

Z-Viper

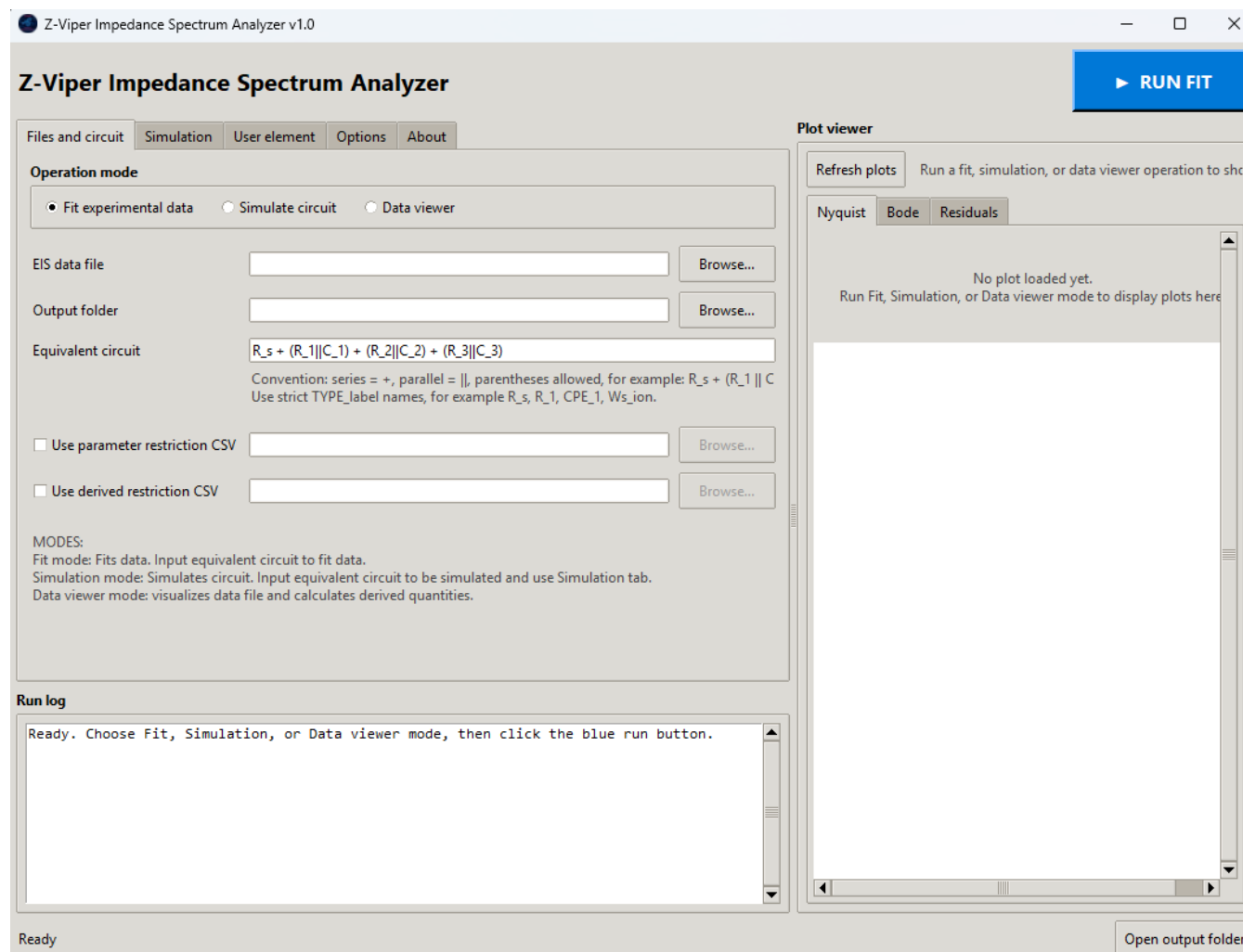
Impedance Spectrum Analyzer

v1.0

User Manual

Copyright ©2026

The contents of this document are provided "as is". The author makes no representations or warranties with respect to the accuracy or completeness of the contents of this publication and reserves the right to make changes to specifications and product descriptions at any time without notice.



Contents

1. Installation and launch	3
2. GUI structure	3
3. Operation modes	5
4.1 Fit experimental data	5
4.2 Simulation.....	8
4.3 Data Viewer	8
4. Equivalent-circuit syntax	9
5. User-defined element tab	10
6. Output Options.....	11
7. Run Log	12
8. Recommended practical workflow.....	12
9. Troubleshooting	12

1. Installation and launch

Using the .exe file:

Download the Z-Viper_Impedance_Spectrum_Analyser_v1.0.exe file.

Run the .exe file.

2. GUI structure

The software is divided into different tabs:

- **Files and circuit:** choice of the operation mode (see [Section 3](#)), path to data file and output folder, input of the equivalent circuit (see [Section 4](#)), and input of optional fit restriction files.

The screenshot shows the 'Files and circuit' tab of the Z-Viper software. It features a 'Simulation' sub-tab and an 'Operation mode' section with three radio buttons: 'Fit experimental data' (selected), 'Simulate circuit', and 'Data viewer'. Below this, there are input fields for 'EIS data file' (C:/PATH/Data.txt), 'Output folder' (C:/PATH/Output), and 'Equivalent circuit' (R_s + (R_1 || C_1) + (R_2 || C_2) + (R_3 || C_3)). A convention note explains the notation: series = +, parallel = ||, parentheses allowed, for example: R_s + (R_1 || C_1 || C_2) || C_3. There are also checkboxes for 'Use parameter restriction CSV' and 'Use derived restriction CSV', each with a 'Browse...' button. At the bottom, a 'MODES' section explains the three modes: Fit mode (fits data), Simulation mode (simulates circuit), and Data viewer mode (visualizes data).

- **Simulation:** to be used when running the Simulation mode (see [Section 3.2](#))

The screenshot shows the 'Simulation' sub-tab within the 'Files and circuit' tab. It contains instructions: 'Simulation mode uses the circuit from the Files and circuit tab. First run without a simulation parameter CSV to create simulation_parameters_template.csv. Then edit the value column, select the edited CSV here, and run again to generate simulated_data.csv and plots.' There is a checkbox for 'Use simulation parameters CSV' with a 'Browse...' button. Below this, there are three input fields: 'Simulation frequency min Hz' (0.1), 'Simulation frequency max Hz' (1000000.0), and 'Number of simulation points' (120).

- **User element:** define up to three different user-defined circuit elements (see [Section 5](#)).

Define up to three user-defined U elements without editing Python code.
 Use the enabled element names in the circuit, for example: $R_s + U_1 + U_2$.
 Each expression uses local variables f [Hz], ω or w [rad/s], and $P1...P5$.
 The imaginary expression is $\text{Im}(Z)$, not $-\text{Im}(Z)$. Parameter values/bounds are edited in the generated CSV templates.

Use	U element name	# P	Re(Z) expression	Im(Z) expression	Example parameter names
☒	U_1	1	P1	0	P1_1
☐	U_2	1	P1	0	P1_2
☐	U_3	1	P1	0	P1_3

Examples:
 Pure resistor: $\text{Re}(Z)=P1$, $\text{Im}(Z)=0$
 Capacitor-like: $\text{Re}(Z)=0$, $\text{Im}(Z)=-1/(\omega \cdot P1)$
 Series R+C: $\text{Re}(Z)=P1$, $\text{Im}(Z)=-1/(\omega \cdot P2)$
 Debye-like relaxation: $\text{Re}(Z)=P1/(1+(\omega \cdot P2)^2)$, $\text{Im}(Z)=-P1 \cdot \omega \cdot P2 / (1+(\omega \cdot P2)^2)$

Allowed functions: sqrt , exp , log , log10 , sin , cos , tanh , abs , minimum , maximum , where .

Use enabled names exactly in the circuit: U_1 , U_2 , U_{ion} , etc.
 Each U element uses local $P1...P5$ in its expression, but the CSV parameters are automatically namespaced, e.g. $P1_1$, $P2$.

- **Options:** fitting ([Section 3.1](#)) and output options ([Section 6](#)).

Fit Options

Residual weighting: **modulus** Robust loss: **soft_l1**

F scale: **0.05** Random seed: **42**

Max function evaluations: **10000** Random starts: **20**

Frequency window

Frequency min Hz blank = **all** Frequency max Hz blank = **all**

Output

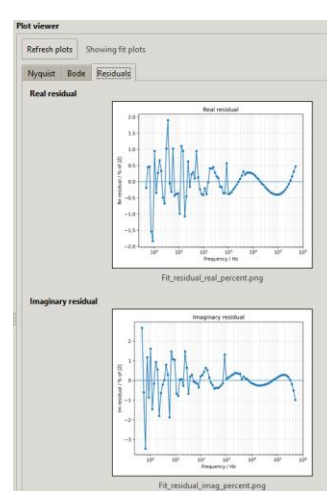
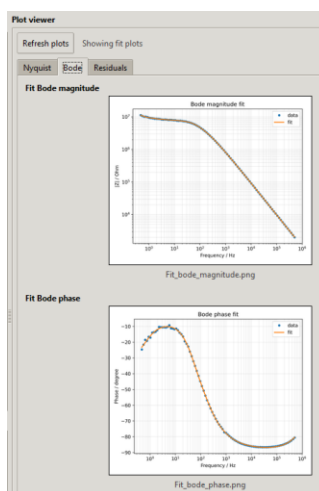
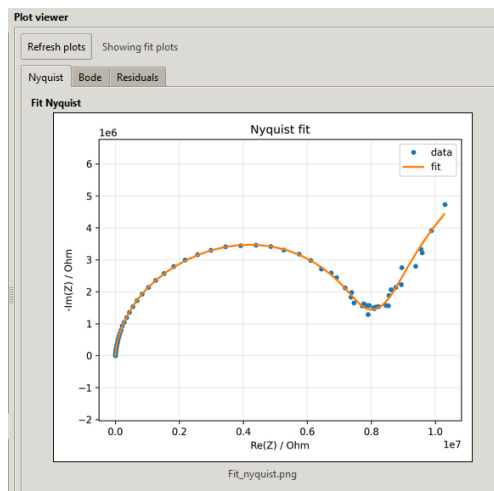
☒ Save output files

Plot DPI: **300**

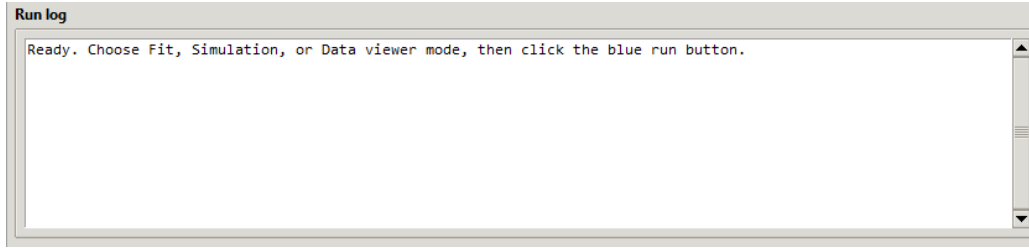
☐ Open saved plot images after run

Choose to save the output files (CSVs and PNGs) to the output folder and if you want to automatically open the fig. The PNGs are saved with the specified DPI.

- **Plot viewer:** The Plot viewer displays the generated figures directly inside the GUI after a successful run. It is organized into separate tabs for Nyquist, Bode, and residual plots, allowing the user to quickly visually inspect the output. The **Refresh plots** button reloads the figures from the selected output folder, which is useful if plots were regenerated or updated during a new run.



- **Run Log:** displays real-time progress, warnings, errors, output-file messages, fit metrics, and final parameter values so the user can verify and troubleshoot each run.



3. Operation modes

4.1 Fit experimental data

Fits an experimental impedance spectrum with the selected equivalent circuit, extracting best-fit element values, residuals, fit metrics, parameter tables, and Nyquist/Bode/residual plots.

Input

- **Data file** (.txt) The standard experimental input format is:

line 1: number of points

line 2 onward: $\text{Re}(Z)$, $-\text{Im}(Z)$, Frequency

The program assumes the second column is $-\text{Im}(Z)$, which is positive for normal capacitive arcs.

- **Output folder:** Folder where all the output files will be saved
- **Circuit string:** String defining the equivalent circuit. See [Section 4](#).

Output

File	Description
Fit_data.csv	Contains the measured data, fitted impedance values, impedance magnitude/phase, and real/imaginary residuals at each frequency point.
Fit_parameters.csv	Contains the final fitted parameter values together with their initial values, bounds, vary/fixed status, units, and descriptions.
Fit_parameter_restrictions_template.csv	Template file listing fit parameters, starting values, hard bounds, vary/fixed status, units, and descriptions. <i>This file is generated only if the "Use parameter restriction CSV" option is NOT selected.</i>
Fit_derived_restrictions_template.csv	Template file for optional derived physical constraints, such as branch peak frequencies, Warburg transition frequencies, and parameter ratios. <i>This file is generated only if the "Use derived restriction CSV" option is NOT selected.</i>
Fit_nyquist.png	Nyquist plot showing the experimental impedance data together with the fitted circuit response.
Fit_bode_magnitude.png	Bode magnitude plot showing experimental and fitted $ Z $ as a function of frequency.
Fit_bode_phase.png	Bode phase plot showing experimental and fitted phase angle as a function of frequency.
Fit_residual_imag_percent.png	Plot of the imaginary-part residual as a percentage of measured $ Z $ versus frequency.
Fit_residual_real_percent.png	Plot of the real-part residual as a percentage of measured $ Z $ versus frequency.
Fit_model_summary.txt	Summary of the fitted circuit, data file, active settings, optimizer result, fit metrics, and best-fit parameters.
Fit_optimizer_history.csv	Records the outcome of each optimizer start, including cost, success flag, number of function evaluations, and optimizer message.
Fit_metrics.csv	Summarizes the goodness-of-fit values, including RMSE, relative RMSE, chi-square, AIC, and BIC.

Fitting procedure

1. Select **Fit experimental data** (Files and circuit tab).
2. Select the data file.
3. Select an output folder.
4. Enter the equivalent circuit (see [Section 4](#)).
5. Optionally select a parameter restriction CSV.
6. Optionally select a derived restriction CSV.
7. Choose settings in Fit options.
8. Click RUN FIT.

During fitting, the program writes templates, builds default parameter guesses, applies restrictions, runs nonlinear least-squares from multiple starts, and saves CSV outputs and plots.

Parameter restriction CSV

The program writes *parameter_restrictions_template.csv* if no file is selected. Edit it and select it as the parameter restriction CSV for later fits.

Column	Meaning
parameter	Parameter name, optionally with unit in brackets.
value	Initial value used by the optimizer.
min	Hard lower bound.
max	Hard upper bound.
vary	True = fit this parameter; False = fix this parameter.
description	Human-readable explanation.

Example:

```
parameter,value,min,max,vary,description
R_s [Ohm],250,100,1000,True,Series/contact resistance
R_1 [Ohm],1.0E6,1.0E4,1.0E8,True,Resistance
Q_1 [S*s^n],1.0E-9,1.0E-12,1.0E-6,True,CPE Q parameter
n_1 [-],0.90,0.50,1.00,True,CPE exponent
```

The value column is the starting guess, not the final value. If vary is False, the parameter is fixed at value.

Important note: More complex equivalent circuits can have multiple “minima” when fitted, leading to good fits but with unphysical values for the parameters. Thus, it might be important to restrict the parameters to physically meaningful values.

Derived/physical restriction CSV

The program writes *derived_restrictions_template.csv* if no file is selected. Derived restrictions are soft penalties applied to calculated physical quantities rather than direct parameters.

Type	Meaning
RCPE_PEAK_HZ	Peak/characteristic frequency of an R CPE branch.
RC_PEAK_HZ	Peak frequency of an R C branch.
RL_TRANSITION_HZ	Transition frequency of an R L branch.
RLPE_TRANSITION_HZ	Transition estimate for an R LPE branch.
L_TRANSITION_HZ	Transition estimate for an L LPE branch.
WS_TRANSITION_HZ	Approximate transition frequency of a short Warburg.
WS_LF_RESISTANCE_OHM	Low-frequency resistance scale of a short Warburg.
WO_TRANSITION_HZ	Approximate transition frequency of an open Warburg.
WO_LF_RESISTANCE_OHM	Low-frequency resistance scale of an open Warburg.
PARAMETER_RATIO	Ratio between two parameters.

Important columns are enabled, name, type, parameter-name columns such as R/Q/n/Wsr/Wsc, min, max, weight, and note. Disabled rows are ignored.

Important note: This restriction constrains physical quantities calculated from fitted parameters, not the parameters themselves. This is particularly useful to avoid “branch swapping” which avoids confusion when comparing the same device under different conditions. This is optional and to be used only at later stages of the fit.

Fit options and optimization

The fit options are found in the [Options](#) tab.

Residual weighting

Option	Meaning	Recommended use
modulus	Residual divided by $ Z $ point-by-point.	Recommended default for EIS.
sqrt_modulus	Residual divided by $\sqrt{ Z }$.	Intermediate weighting.
none	Unweighted residual in ohms.	Use only when absolute-ohm errors should dominate.

Robust loss

Loss	Behavior	Recommendation
linear	Ordinary least squares; large residuals dominate strongly.	Strict comparison only.
soft_l1	Smooth robust loss; large residuals are downweighted gradually.	Best default.
huber	Least squares for small residuals, linear penalty for large residuals.	Good robustness cross-check.
cauchy	Strongly suppresses large residuals.	Diagnostic only.
arctan	Very aggressive limiting of very large residuals.	Stress test only.

F Scale

F_SCALE controls where robust loss starts treating residuals as large. With modulus weighting, 0.05 roughly corresponds to a 5% normalized residual transition. Larger values are closer to ordinary least squares; smaller values are more robust. F_SCALE is ignored when LOSS = linear.

F scale	Effect
0.10	Closer to ordinary least squares; less robust.
0.05	Good default.
0.03	More robust; downweights large residuals earlier.
0.01	Very aggressive; can hide real low-frequency behavior.

Random starts and random seed

N_RANDOM_STARTS controls how many randomized initial guesses are tried in addition to the main starting guess. RANDOM_SEED makes those random starting guesses reproducible. If different seeds give nearly the same result, the solution is stable with respect to initialization. If different seeds give significantly different results, the model is non-unique or the bounds need improvement.

Recommended defaults

```

WEIGHT_MODE = modulus
LOSS = soft_l1
F_SCALE = 0.05
N_RANDOM_STARTS = 20 to 50
RANDOM_SEED = 42
MAX_NFEV = 10000 to 30000
DPI = 300

```

For robustness checking, use LOSS = huber with F_SCALE = 0.05. For a more least-squares-like check, use LOSS = soft_l1 with F_SCALE = 0.10. Use cauchy only as an outlier diagnostic.

4.2 Simulation

Generates an impedance spectrum from the selected equivalent circuit using values supplied in a simulation-parameter CSV file.

Input:

- **Simulation parameters** (.csv). This file gives the parameters for the circuit elements. If no file is input, the program will generate a template the user can modify.
- **Circuit string**: String defining the equivalent circuit. See section X
- **Output folder**: Folder where all the output files will be saved

Output:

File	Description
Simulation_parameters_template.csv	Template file listing all circuit parameters needed for simulation; edit the value column and use it as the simulation-parameter input file. <i>Automatically generated (only) if no simulation parameters file is input.</i>
Simulation_data.csv	Contains the simulated impedance values at each frequency, including $\text{Re}(Z)$, $\text{Im}(Z)$, $-\text{Im}(Z)$, $ Z $, phase, and classic export columns.
Simulation_nyquist.png	Nyquist plot of the simulated impedance spectrum, showing $\text{Re}(Z)$ versus $-\text{Im}(Z)$.
Simulation_bode_magnitude.png	Bode magnitude plot of the simulated impedance spectrum, showing simulated $ Z $ versus frequency.
Simulation_bode_phase.png	Bode phase plot of the simulated impedance spectrum, showing simulated phase angle versus frequency.

Simulation procedure

1. Select **Simulation** (Files and circuit tab).
2. Select an output folder.
3. Enter the equivalent circuit (see [Section 4](#)).
4. Run program to generate Simulation parameters template file (optional)
5. Select Simulation parameters CSV file ([Simulation](#) tab).
6. Click RUN SIMULATION.

4.3 Data Viewer

Loads an experimental impedance data file, calculates derived quantities such as $|Z|$ and phase, and displays/saves Nyquist and Bode plots without fitting a model.

Input:

- **Data file** (.txt)
- **Output folder**: Folder where all the output files will be saved

Output:

File	Description
View_Data.csv	Contains the loaded experimental impedance spectrum data with derived quantities such as $\text{Re}(Z)$, $\text{Im}(Z)$, $-\text{Im}(Z)$, $ Z $, phase, and classic export columns.
View_Data_Nyquist.png	Nyquist plot of the experimental data, showing $\text{Re}(Z)$ versus $-\text{Im}(Z)$ without any model fit.
View_Data_Bode_Magnitude.png	Bode magnitude plot of the experimental data, showing measured $ Z $ versus frequency.
View_Data_Bode_Phase.png	Bode phase plot of the experimental data, showing measured phase angle versus frequency.

View Data procedure

7. Select **Data Viewer** (Files and circuit tab).
8. Select data file.
9. Select an output folder.
10. Click VIEW DATA.

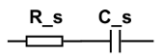
4. Equivalent-circuit syntax

Operator	Meaning	Example
+	Series connection	$R_s + R_1$
	Parallel connection	$R_1 CPE_1$
(...)	Grouping	$R_s + (R_1 CPE_1)$

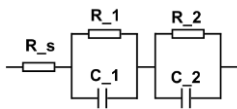
Spaces do not matter. Element names must be written as **TYPE_label1**. Bare names such as Rs, R1, CPE1, and WsIon are intentionally rejected. Valid examples: R_s , R_1 , C_1 , CPE_1 , Ws_{ion} , U_{custom}

Example Circuits:

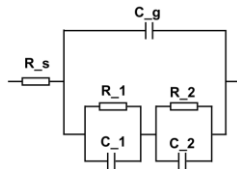
$R_s + C_s$



$R_s + (R_1 || C_1) + (R_2 || C_2)$



$R_s + (C_g || ((R_1 || C_1) + (R_2 || C_2)))$



Supported elements and parameters

Element TYPE	Meaning	Parameters (unit)
R	Resistor	$R (\Omega)$
C	Ideal capacitor	$C (F)$
L	Ideal inductor	$L (H)$
CPE	Constant phase element	$Q (S \cdot s^n)$, $n (-)$
LPE	Inductive phase element	QL_label , nL_label
W	Semi-infinite Warburg	$W (\Omega/s^{1/2})$
Ws	Finite-length short/transmissive Warburg	$Wsr (\Omega/s^{1/2})$, $Wsc (s^{1/2})$
Wo	Finite-length open/reflective Warburg	$Wor (\Omega/s^{1/2})$, $Woc (s^{1/2})$
G	Gerischer element	$Yg (S \cdot s^{-1/2})$, $Kg (s^{-1})$
U	User-defined element	Default: $UP1_label$, $UP2_label$,

Element TYPE	Mathematical convention
R	$Z = R$
C	$Z = 1/(j\omega C)$
L	$Z = j\omega L$
CPE	$Z = Q^{-1}(j\omega)^{-n}$
LPE	$Z = QL(j\omega)^{nL}$
W	$Z = W(1-j)/\sqrt{\omega}$
Ws	$Z = Wsr(1-j)/\sqrt{\omega} * \tanh(Wsc\sqrt{j\omega})$
Wo	$Z = Wor(1-j)/\sqrt{\omega} * \coth(Woc\sqrt{j\omega})$
G	$Z = 1/(Yg\sqrt{Kg + j\omega})$
U	

Note: In this convention Wsr/Wor are Warburg coefficients and $Wsc/Woc = \delta/\sqrt{D}$, where δ is the diffusion length and D is the diffusion coefficient.

For the User Defined element see [Section 5](#).

5. User-defined element tab

The User element tab allows custom impedance elements to be defined without editing the Python source code. The program supports up to three GUI-defined user elements. Use strict names such as U_1, U_2, U_fast, U_ion, or U_custom. Do not use U1 or Ufast.

Expression variables

For each user element you enter the element name, number of parameters, Re(Z) expression, and Im(Z) expression. The imaginary expression is Im(Z), not -Im(Z).

Available variables:

f ordinary frequency in Hz
omega angular frequency in rad/s
w same as omega
P1...P5 local user-element parameters

Available functions include:

sqrt, exp, log, log10, sin, cos, tan, sinh, cosh, tanh, abs, minimum, maximum, where

Parameter naming

Circuit element	Local variables	CSV parameter names
U_1	P1, P2	P1_1, P2_1
U_2	P1, P2, P3	P1_2, P2_2, P3_2
U_ion	P1, P2	P1_ion, P2_ion
U_trap	P1	P1_trap

Examples

Pure resistor user element:

Element name: U_res
Number of parameters: 1
Re(Z): P1
Im(Z): 0
Circuit: R_s + U_res

Capacitor-like user element:

Element name: U_cap
Number of parameters: 1
Re(Z): 0
Im(Z): $-1/(\omega * P1)$
Circuit: R_s + U_cap

Two user elements in one circuit:

U_1:
Number of parameters: 2
Re(Z): P1
Im(Z): $-1/(\omega * P2)$

U_2:
Number of parameters: 1
Re(Z): P1
Im(Z): 0

Circuit: R_s + U_1 + U_2

Practical cautions

- Use physically motivated formulas.
- Test user elements first in Simulation mode.
- Use the fewest parameters needed.
- Apply sensible bounds in fit mode.
- Check stability against random seed and robust loss settings.
- Do not use multiple high-parameter U elements unless there is a clear physical reason.

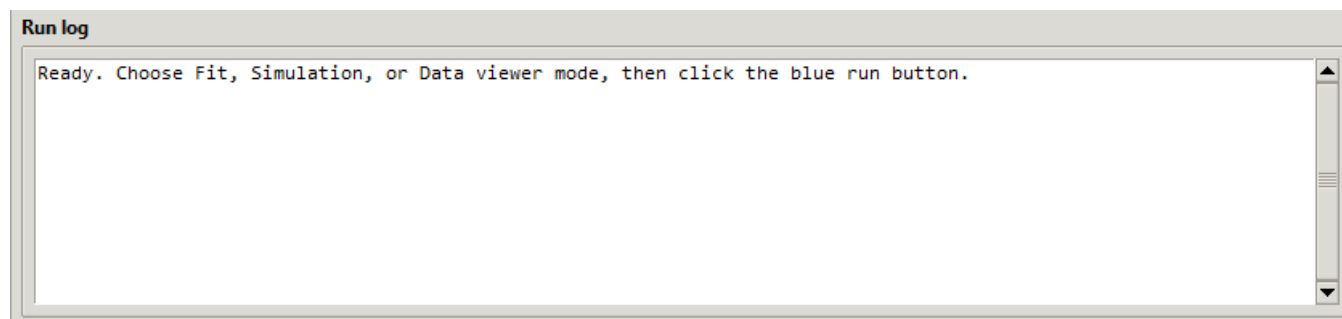
6. Output Options

The Output options control how the program saves and displays the results generated by each mode. They let the user decide whether CSV/PNG files should be written to the selected output folder, set the resolution of saved plot images through the DPI value, and optionally open the saved figures automatically after a run. These settings do not change the fitting, simulation, or data-viewer calculations themselves; they only control how the results are exported and presented.

- **Save output files:** When checked, the program writes the generated CSV files and PNG plots to the selected output folder; when unchecked, the run can be used mainly for viewing results in the GUI without saving new output files.
- **Plot DPI:** Sets the resolution of the saved PNG plots; higher values give sharper figures for reports/papers but larger file sizes, while 300 DPI is a good default for publication-quality images.
- **Open saved plot images after run:** When checked, the program automatically opens the saved PNG plots in the operating-system image viewer after the run finishes; the plots are still shown inside the GUI either way.
- **Open output folder** button is present on the bottom right of the GUI window. Click it to open the currently selected output folder.

7. Run Log

The Run log window shows the real-time text output generated while the program is running. It reports the selected operation mode, loaded files, parsed circuit elements, active restriction files, fitting progress, warnings, errors, generated output files, fit metrics, and final parameter values. It is mainly used for checking that the run used the intended settings and for diagnosing problems if the fit, simulation, or data-viewer operation fails.



8. Recommended practical workflow

1. Use Data viewer mode to check that the file loads correctly and the raw plots look sensible.
2. Run Fit mode once without restriction CSVs to generate templates.
3. Edit `parameter_restrictions_template.csv` to set physically reasonable values and bounds.
4. Optionally edit `derived_restrictions_template.csv` to constrain branch frequencies or Warburg scales.
5. Fit again using the edited CSV files.
6. Inspect Nyquist/Bode overlays, residuals, metrics, and whether parameters hit bounds.
7. Rerun with different random seeds and with huber loss as a robustness check.
8. Use Simulation mode to verify how parameter changes affect the expected spectrum.

9. Troubleshooting

Problem	Likely cause	Fix
Circuit parsing error	Element names do not follow TYPE_label.	Use names like R_1, CPE_1, Ws_ion.
No numeric rows found	File format is not Re(Z), -Im(Z), Frequency.	Check columns, delimiters, and header lines.
Fit gives crazy values	Bounds too loose or branch swapping.	Use parameter and derived restriction CSVs.
Fit changes strongly with random seed	Multiple local minima or non-identifiability.	Tighten bounds, add derived restrictions, increase random starts.
Parameters hit bounds	Bounds may be too tight or model is forcing a limit.	Check whether the bound is intentional.
Simulation only writes a template	No simulation parameter CSV was selected.	Edit and select <code>simulation_parameters_template.csv</code> .
Low-frequency region fits poorly	Drift, non-stationarity, missing physics, or weakly identifiable Warburg branch.	Compare robust losses, inspect residuals, consider a frequency window or modified circuit.